

虚拟内存的运作需要硬件和操作系统软件之间精密复杂的交互，包括对处理器生成的每个地址的硬件翻译。基本思想是把一个进程虚拟内存的内容存储在磁盘上，然后用主存作为磁盘的高速缓存。第9章将解释它如何工作，以及为什么对现代系统的运行如此重要。

1.7.4 文件

文件就是字节序列，仅此而已。每个 I/O 设备，包括磁盘、键盘、显示器，甚至网络，都可以看成是文件。系统中的所有输入输出都是通过使用一小组称为 Unix I/O 的系统函数调用读写文件来实现的。

文件这个简单而精致的概念是非常强大的，因为它向应用程序提供了一个统一的视图，来看待系统中可能含有的所有各式各样的 I/O 设备。例如，处理磁盘文件内容的应用程序可以非常幸福，因为他们无须了解具体的磁盘技术。进一步说，同一个程序可以在使用不同磁盘技术的不同系统上运行。你将在第10章中学习 Unix I/O。

旁注 Linux 项目

1991年8月，芬兰研究生 Linus Torvalds 谨慎地发布了一个新的类 Unix 的操作系统内核，内容如下。

来自：torvalds@klaava.Helsinki.FI(Linus Benedict Torvalds)

新闻组：comp.os.minix

主题：在 minix 中你最想看到什么？

摘要：关于我的新操作系统的小调查

时间：1991年8月25日 20:57:08 GMT

每个使用 minix 的朋友，你们好。

我正在做一个(免费的)用在 386(486)AT 上的操作系统(只是业余爱好，它不会像 GNU 那样庞大和专业)。这个想法自4月份就开始酝酿，现在快要完成了。我希望得到各位对 minix 的任何反馈意见，因为我的操作系统在某些方面与它相类似(其中包括相同的文件系统的物理设计(因为某些实际的原因))。

我现在已经移植了 bash(1.08)和 gcc(1.40)，并且看上去能运行。这意味着我需要几个月的时间来让它变得更实用一些，并且，我想要知道大多数人想要什么特性。欢迎任何建议，但是我无法保证我能实现它们。:-)

Linus (torvalds@kruuna.helsinki.fi)

就像 Torvalds 所说的，他创建 Linux 的起点是 Minix，由 Andrew S. Tanenbaum 出于教育目的开发的一个操作系统[113]。

接下来，如他们所说，这就成了历史。Linux 逐渐发展成为一个技术和文化现象。通过和 GNU 项目的力量结合，Linux 项目发展成了一个完整的、符合 Posix 标准的 Unix 操作系统的版本，包括内核和所有支撑的基础设施。从手持设备到大型计算机，Linux 在范围如此广泛的计算机上得到了应用。IBM 的一个工作组甚至把 Linux 移植到了一块腕表中！

1.8 系统之间利用网络通信

系统漫游至此，我们一直是把系统视为一个孤立的硬件和软件的集合体。实际上，现

代系统经常通过网络和其他系统连接到一起。从一个单独的系统来看，网络可视为一个 I/O 设备，如图 1-14 所示。当系统从主存复制一串字节到网络适配器时，数据流经过网络到达另一台机器，而不是比如说到本地磁盘驱动器。相似地，系统可以读取从其他机器发送来的数据，并把数据复制到自己的主存。

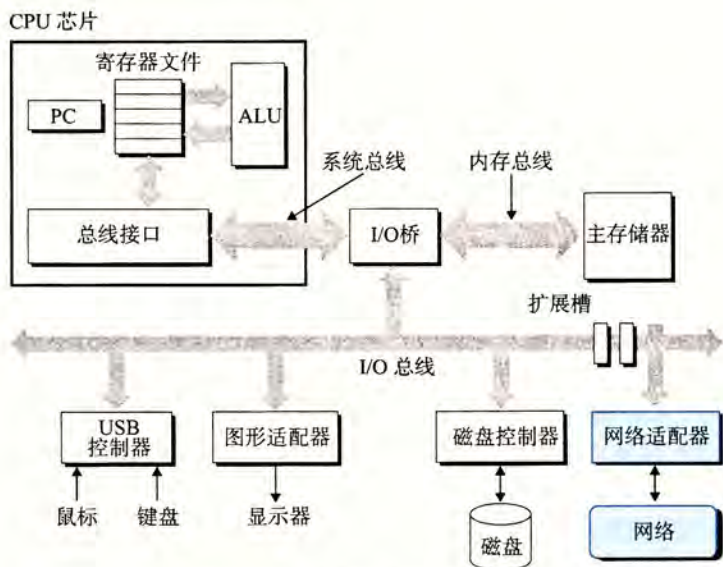


图 1-14 网络也是一种 I/O 设备

随着 Internet 这样的全球网络的出现，从一台主机复制信息到另外一台主机已经成为计算机系统最重要的用途之一。比如，像电子邮件、即时通信、万维网、FTP 和 telnet 这样的应用都是基于网络复制信息的功能。

回到 hello 示例，我们可以使用熟悉的 telnet 应用在一个远程主机上运行 hello 程序。假设用本地主机上的 telnet 客户端连接远程主机上的 telnet 服务器。在我们登录到远程主机并运行 shell 后，远端的 shell 就在等待接收输入命令。此后在远端运行 hello 程序包括如图 1-15 所示的五个基本步骤。



图 1-15 利用 telnet 通过网络远程运行 hello

当我们在 telnet 客户端键入“hello”字符串并敲下回车键后，客户端软件就会将这个字符串发送到 telnet 的服务器。telnet 服务器从网络上接收到这个字符串后，会把它传递给远端 shell 程序。接下来，远端 shell 运行 hello 程序，并将输出返回给 telnet 服务器。最后，telnet 服务器通过网络把输出串转发给 telnet 客户端，客户端就将输出串输出到我们的本地终端上。

这种客户端和服务端之间交互的类型在所有的网络应用中是非常典型的。在第 11 章中，你将学会如何构造网络应用程序，并利用这些知识创建一个简单的 Web 服务器。

1.9 重要主题

在此，小结一下我们旋风式的系统漫游。这次讨论得出一个很重要的观点，那就是系统不仅仅只是硬件。系统是硬件和系统软件互相交织的集合体，它们必须共同协作以达到运行应用程序的最终目的。本书的余下部分会讲述硬件和软件的详细内容，通过了解这些详细内容，你可以写出更快速、更可靠和更安全的程序。

作为本章的结束，我们在此强调几个贯穿计算机系统所有方面的重要概念。我们会在本书中的多处讨论这些概念的重要性。

1.9.1 Amdahl 定律

Gene Amdahl，计算领域的早期先锋之一，对提升系统某一部分性能所带来的效果做出了简单却有见地的观察。这个观察被称为 Amdahl 定律(Amdahl's law)。该定律的主要思想是，当我们对系统的某个部分加速时，其对系统整体性能的影响取决于该部分的重要性和加速程度。若系统执行某应用程序需要时间为 T_{old} 。假设系统某部分所需执行时间与该时间的比例为 α ，而该部分性能提升比例为 k 。即该部分初始所需时间为 αT_{old} ，现在所需时间为 $(\alpha T_{old})/k$ 。因此，总的执行时间应为

$$T_{new} = (1 - \alpha)T_{old} + (\alpha T_{old})/k = T_{old}[(1 - \alpha) + \alpha/k]$$

由此，可以计算加速比 $S = T_{old}/T_{new}$ 为

$$S = \frac{1}{(1 - \alpha) + \alpha/k} \quad (1.1)$$

举个例子，考虑这样一种情况，系统的某个部分初始耗时比例为 60% ($\alpha = 0.6$)，其加速比例因子为 3 ($k = 3$)。则我们可以获得的加速比为 $1/[0.4 + 0.6/3] = 1.67$ 倍。虽然我们对系统的一个主要部分做出了重大改进，但是获得的系统加速比却明显小于这部分的加速比。这就是 Amdahl 定律的主要观点——要想显著加速整个系统，必须提升全系统中相当大的部分的速度。

旁注 表示相对性能

性能提升最好的表示方法就是用比例的形式 T_{old}/T_{new} ，其中， T_{old} 为原始系统所需时间， T_{new} 为修改后的系统所需时间。如果有所改进，则比值应大于 1。我们用后缀“ $\times$ ”来表示比例，因此，“ $2.2\times$ ”读作“2.2 倍”。

表示相对变化更传统的方法是用百分比，这种方法适用于变化小的情况，但其定义是模糊的。应该等于 $100 \cdot (T_{old} - T_{new})/T_{new}$ ，还是 $100 \cdot (T_{old} - T_{new})/T_{old}$ ，还是其他的值？此外，它对较大的变化也没有太大意义。与简单地说性能提升 $2.2\times$ 相比，“性能提升了 120%”更难理解。



练习题 1.1 假设你是个卡车司机，要将土豆从爱达荷州的 Boise 运送到明尼苏达州的 Minneapolis，全程 2500 公里。在限速范围内，你估计平均速度为 100 公里/小时，整个行程需要 25 个小时。

- 你听到新闻说蒙大拿州刚刚取消了限速，这使得行程中有 1500 公里卡车的速度可以为 150 公里/小时。那么这对整个行程的加速比是多少？
- 你可以在 www.fasttrucks.com 网站上为自己的卡车买个新的涡轮增压器。网站现货供应各种型号，不过速度越快，价格越高。如果想要让整个行程的加速比为 $1.67\times$ ，那么你必须以多快的速度通过蒙大拿州？